

New model implementation plan

Source: `docs/New-model/* .docx` (Brand Portal, Packages Details, Survey, and the 24 Aug 2026 standup notes) plus a full-repo gap analysis against them, done 26 Aug 2026. This is the working plan for closing the gap between what those docs describe and what's in the codebase today — **nothing below is built yet** except where a section says otherwise. Update this file as phases land, the way `docs/database-migrations.md` tracks migrations.

Companion doc: `docs/survey-form-builder-design.md` covers the survey/quality- engine build in full — this plan only summarises where it sits in the sequence.

1. What the 24 Aug standup already decided

Treat these five as settled requirements, not open questions:

Decision	Detail
Package pricing structure	Starter £2,500, Growth £3,750, Premium £5,000, Enterprise Custom from £6,000 minimum (multiples of £500)
Admin-only campaign creation	Brands submit an intake request; only admin creates the <code>campaigns</code> row and links brand↔artist
Artist payment release condition	Remainder of artist fee pays out only after the artist submits a report verifying ticket sales
Event change validity period	Sponsors may request an event/artist change up to 2 days before the event date
Survey tool	Build in-house, not Typeform/Jotform — see <code>docs/survey-form-builder-design.md</code>

Two more things surfaced since, not from the standup transcript but load-bearing for the same plan:

- **Direct brand↔artist chat is being removed.** Admin is the required intermediary for anything cross-party now, matching "any requests by brands to change their budget or campaign details will be handled through administrative intervention and direct conversation with the team." See §6.
- **The Brand Portal doc's checkout language conflicts with the agreed payments model.** See §7 — this blocks Phase 6 until resolved.

2. Phase 1 — Admin-only campaign creation

Current state: `createCampaign` in `src/app/dashboard/campaigns/actions.ts:104-156` lets the brand insert directly into `campaigns`; there's no review step. Admin's only campaign tooling today is `src/app/dashboard/admin/campaigns/page.tsx`

- `MatchForm.tsx` — filtering, status changes, and matching an *already-created* campaign to an `event_listings` row. There's no admin "create this campaign for a brand" flow.

Change:

1. Add a `pending_review` value to `campaign_status` (currently `in_progress` | `closed` | `completed`, `src/lib/constants.ts:128-132`) — additive enum change.
2. Rework the brand-facing form into an **intake request**: it still writes a row (so nothing is lost if admin is slow), but lands in `pending_review` and is read-only to the brand until admin acts.
3. Add fields the new intake form collects that the current one doesn't: approx. surveys desired, event category (separate from artist/target category), campaign month/year (today only a free-text `preferred_timeline`), eligibility criteria, brand messaging / market- research area, multi-asset upload (today: `single_image_url`), and a structured artist/event nomination (name + URL as paired fields, not the current free-text `suggested_event_note/url`).
4. Build the missing admin screen: promote a `pending_review` intake into a live campaign — assign a package tier (\$3), link the artist/event, and flip status to `in_progress`. This is net-new UI, not an extension of `MatchForm`.
5. `campaign.created` and `admin.campaign_request` (both already seeded, `0006_notifications.sql`) are the right hooks to reuse for "your request was received" / "a new intake needs review" — no new notification keys needed for this phase.

3. Phase 2 — Package/tier pricing model

Current state: `src/lib/constants.ts:152-211` has no package concept at all. `computePlatformFee()` takes a gross budget and charges the greater of £315 or 9%, +20% VAT; `MIN_SPONSORSHIP_BUDGET_GBP` is £378; `netSponsorshipBudget()` is gross minus that fee. Brands today type an arbitrary `budget_gbp`. The closest existing shape to a "package" is `src/app/dashboard/admin/plans/*` — a generic subscription-plan CRUD (`price_gbp`, `billing_interval`, `features[]`) against the unrelated `plans` table. Structurally reusable (name / price / features / sort order / is-active) but wired to subscriptions, not sponsorships.

Open decision — resolve before building this phase: do the £2,500 / £3,750 / £5,000 package prices *replace* the fee formula (i.e. that price is the full amount the brand pays), or do they feed into it as a gross budget the way Enterprise Custom's "multiples of £500, min £6,000" reads? The standup phrasing ("established with a 2,500 starter tier...") reads as fixed package prices, which would mean Starter/Growth/Premium bypass `computePlatformFee()` entirely while Enterprise Custom keeps using it. Get this confirmed — it changes whether Phase 2 touches `constants.ts`'s money functions or only adds a new table alongside them.

Also open: the benefit matrix. `Packages Details.docx` lists ~10–15 line items under each of 8 categories (Reach, Targeting, Research, Engagement, Brand Awareness, Emotional Brand Impact, Insights, Measurement, Campaign Management) but never states *which tier gets which items* — the Brand Portal doc just says "include a benefit matrix according to the budget (see other page)" without the matrix itself existing anywhere. Someone needs to produce that tier×benefit grid before the pricing page can render it.

Change (once both are resolved):

1. New `campaign_packages` table: tier name, price, participant count, survey count, sort order, is-active — modelled on `plans`' existing shape rather than reusing the `plans` table itself (different domain, same pattern).
2. New `campaign_package_benefits` table (or a `jsonb` column) holding the tier→benefit grid, once it exists.
3. Brand-facing "Campaign Packages" step in the intake form (Phase 1) picks a tier; Enterprise Custom keeps a free-entry budget field with the existing £500-multiple / £6,000-minimum validation, auto-calculating survey count.
4. If Starter/Growth/Premium bypass the fee formula (pending the decision above), `computePlatformFee()` / `netSponsorshipBudget()` stay as-is for Enterprise Custom and any sponsored-event budget math that still derives from a gross figure.

4. Phase 3 — Survey system

Full design in `docs/survey-form-builder-design.md`. Confirmed net-new: a repo-wide search for "survey" across `src/` and `supabase/migrations/` returns zero hits today. Summary of what this phase adds:

- Schema: `survey_templates`, `survey_questions`, `survey_responses`, `survey_answers`.
- Admin drag-and-drop builder (question types, reordering, live preview).
- Participant-facing renderer + submission flow.
- The Response Quality Engine (9-signal scoring → pass / manual review / reject) that Phase 4's reward gating depends on.

This phase is a **dependency for Phase 4** — reward tiering can't be survey-gated until responses and scores exist.

5. Phase 4 — Reward engine rework

Current state: `sponsored/actions.ts:423-470` (`updateParticipation`) only implements attendance verification. Selection is a random draw (`admin/events/sponsored/[id]/SelectionDrawForm.tsx`, wired through `admin/marketplace-actions.ts:426-430`). Reward release (`admin/marketplace-actions.ts:291-395`) is an admin typing a free-form `reward_amount_gbp` (:317-333) — there's no discount/merch-code generation and nothing in `participations` references a survey step at all.

Change:

1. Replace (or gate) the flat admin-typed amount with the new spec's tiering: the first 50 survey completions on a campaign unlock the pre-event ticket subsidy; the post-event survey and any participant past the first 50 get a discount/merch code only.
2. Add code generation + redemption tracking — new columns or a new `reward_codes` table, since today there's no code concept at all, only a number.
3. Gate release on `survey_responses.quality_status = 'pass'` (Phase 3) rather than admin discretion alone.
4. Update the sponsored-events screen (\$ below) to show a live "surveys remaining" counter and the reward tiers, per the new doc's "Sponsorship Overview" and "Reward Engine" sections.

6. Phase 5 — Sponsored events screen additions

Current state: `sponsored/actions.ts:82-213` (`createSponsoredEvent`) covers event details, budget → `remaining_budget_gbp` via `netSponsorshipBudget`, `reward_rules`, `terms`, `banner/branding` uploads (`ASSET_SLOTS = 5, :16-73`, into `sponsored_event_assets` — one generic bucket with a free-text description).

Missing vs. the new spec:

- Surveys-remaining counter and the `# surveys | participation start date` line from "Sponsorship Overview" (needs Phase 3).
- A structured "Reward Engine" display (first-50 vs. late-participant tiers), not just a `reward_rules` free-text blob.
- **Typed proof-of-terms uploads**, distinct from the generic asset slots: artist social-media-mention proof and onsite-branding proof are each their own upload with their own fulfilment status, not one more image in `sponsored_event_assets`. Reuse the `private-uploads` bucket pattern (signed URLs, `{profileId}/{purpose}/{uuid}.{ext}` keying) already used for chat attachments and ticket proofs — same shape, new `purpose` values.

7. Phase 6 — Payments / checkout / orders (blocked)

Do not build this phase until the conflict below is resolved.

`docs/payments-kyc-strategy.md` (10 Aug standup) is the existing, dated, agreed decision: money never sits on the platform. A sponsor pays by direct **bank transfer against an invoice** quoting the ^{SPE}- reference; identity via Stripe Identity; artist/audience payouts via Wise. Stripe Connect Express was explicitly rejected. Line 6 of that doc: "none of this is implemented yet" — confirmed by this gap analysis, a repo search for `stripe|checkout|invoice` returns zero hits in `src/`.

The Brand Portal doc's "Add to cart, checkout, billing/invoicing, order confirmation email... My Orders Page" reads as self-service e-commerce — a different shape. Neither is built, so this isn't a rollback of a shipped feature, but the two docs cannot both be current. **Raise this at the next standup rather than picking one during implementation.**

Once resolved, this phase is either:

- **(a) Matches the invoice model** — an "Orders" list is a read view over campaigns/sponsorships with their invoice reference and payment status; "checkout" becomes "your request has an invoice, here's the reference," not a card-payment flow.
- **(b) Matches the cart model** — genuine payment collection gets added, which reopens the Stripe Connect Express question `payments-kyc-strategy.md` §2 already rejected for a different reason (escrow / account-freezing risk on sponsorship-sized payments) and needs its own design pass.

`docs/PLATFORM.md:123-131,180-181` still lists Stripe/Stripe Connect in its Planned Integrations table — that's stale relative to `payments-kyc-strategy.md` and should be corrected in the same pass as whichever direction is chosen here, not left to imply two different plans are both live.

8. Phase 7 — Artist payment split

Current state: confirmed net-new. A repo search for `payout|artist_payment|remaining_fee|artist_fee` returns zero hits. `sponsored_events.remaining_budget_gbp` is exclusively the *audience* reward pool (per the architecture doc), not an artist fee — there is no schema field, action, or screen tracking what the platform owes the artist.

Open decision: the standup agreed the *condition* (remainder released only after a ticket-sales report) but not the *split* — what portion is upfront vs. held back. Needs a number before the schema can be finalised (a fixed %, a per-package default, or admin-set per sponsorship).

Change (once the split is defined):

1. New fields/table for the artist's own fee: total, upfront amount, paid status, remainder amount, remainder status.
2. Artist-facing "submit ticket sales report" action — likely a form with a number + optional evidence upload, reusing the `private-uploads` pattern again.
3. Admin action to release the remainder once a report is submitted, following the existing `(prevState, formData) -> State` action shape.
4. This likely depends on Phase 6's payment-rail decision for the actual money movement (Wise payout, per `payments-kyc-strategy.md` §3), but the *tracking* schema and artist-facing report flow can be built ahead of that.

9. Phase 8 — Event change window (2-day cutoff)

Current state: no date-cutoff logic exists anywhere — a repo search for `days?`

`before|days_before|cutoff` returns zero hits. `sponsored/actions.ts:337-368` (`updateSponsoredEvent`) only allows edits while `status === 'in_progress'`; once confirmed, the only path is `contactSupport` (:374-397), which opens an admin conversation with no date check at all.

Change:

1. New "request event change" action on a `confirmed` sponsored event, validated against the event's local date/timezone (reuse `formatEventDateTime()` / `event-time.ts` helpers already used elsewhere, so the 2-day comparison respects the same wall-clock-plus-zone model as the rest of the app rather than a naive UTC diff).
2. Reject the request client- and server-side once inside the 2-day window; route anything that needs to happen anyway through the admin-mediated conversation (§6) rather than a self-service edit.

10. Notifications and docs cleanup

New `notification_events` rows needed across the phases above: survey completion, quality-engine review-needed, reward-released-via-survey, artist-remainder-released, event-change-requested/approved. Existing `campaign.created` and `admin.campaign_request` (0006) already cover Phase 1's intake flow and don't need duplicates.

Docs to update as phases land: `docs/PLATFORM.md` Planned Integrations table (§7 above), `docs/database-migrations.md` (every new migration, per its existing convention), `docs/TEST-CASES.md` (currently has zero references to package/tier/survey — nothing to reconcile, just needs new cases added as each phase ships).

6. Cross-cutting: direct brand↔artist chat is removed

Current state: `src/lib/data/messaging.ts` has two conversation kinds. `getOrCreateConversation()` (:18-58) creates a "**partner**"-kind thread directly between a brand and an artist/organiser — called from `src/app/dashboard/campaigns/actions.ts:258`, `src/app/dashboard/discover/actions.ts:34`, and `src/app/dashboard/admin/marketplace-actions.ts:226`.

`getOrCreateSupportConversation()` (:66-118) creates a "**support**"-kind thread between a user and the admin team, and already supports admin starting a thread with someone who hasn't written in first (`startAdminThread`, `src/app/dashboard/messages/actions.ts:124-151`).

Change: admin is now the required intermediary for everything cross-party, matching the standup's framing that budget/detail changes go through "administrative intervention and direct conversation with the team." Concretely:

1. Remove the three "**partner**"-kind creation call sites above (or repoint them at `getOrCreateSupportConversation` so a brand's "contact organiser" click opens a thread with admin, not with the artist).
2. Decide whether admin **relays manually** between two independent "**support**" threads (brand↔admin, artist↔admin — no data linking them, admin reads one and writes the other) or the schema gains a lighter-weight way to flag "this support thread is about campaign X" so admin isn't hunting for context. The former needs no schema change; the latter is a small additive column on `conversations`.
3. Don't rely on hiding the UI alone — `admin/marketplace-actions.ts:226` already runs admin-side, so it's the one call site that could plausibly stay, but check whether it's still opening a "**partner**" thread

between the two parties or just showing admin the match. If it's the former, change it too, and consider whether RLS/a check constraint should stop "partner"-kind inserts outright rather than leaving it reachable by anyone who calls the (still-exported) function directly.

- 4. `is_my_conversation_peer()` (the RLS policy helper) and the `conversations_unique_thread` constraint (migration 0008) aren't "partner"-specific — audit whether either needs a `kind` condition added once direct threads stop being created, or whether they're fine left as-is because no new "partner" rows will ever be inserted.

7. Still to decide (all in one place)

Question	Blocks
Do package prices replace <code>computePlatformFee()</code> , or feed into it?	Phase 2
What's the tier→benefit matrix? (Not in either doc today.)	Phase 2's pricing page
Cart/checkout self-service, or the agreed bank-transfer/invoice model?	Phase 6, and anything downstream of it
What's the artist upfront/remainder split?	Phase 7
Manual admin relay between two threads, or a lightweight link between them?	§6

8. What isn't changing

Attendance verification, the `registered` → `ticket_uploaded` → `attendance_verified` → `reward_released` ladder's shape, and RLS's owner-scoped-plus-admin-override pattern all stay as they are — this plan adds gates and fields onto that ladder (survey pass, quality score), it doesn't replace it.